impl TradeoffCurve::from_gdp

Michael Shoemate

May 2, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `mu` of type `f64`

Caller-verified

None.

Pseudocode

```
def tradeoffcurve_from_gdp(I: Intent, mu: float) -> TradeoffCurve:
    if not is_finite(mu) or mu < 0.0:
        raise ValueError("mu must be a finite non-negative number")

    sqrt_2_lo = next_down(sqrt(2.0))
    sqrt_2_hi = next_up(sqrt(2.0))
    p_max = next_down_f32(2.0)
    t_max = f32_max
    beta_rounds_up = I.CALIBRATION
    t_rounds_up = not I.CALIBRATION

    def tradeoff(alpha: float) -> float:
        if not is_finite(alpha) or not (0.0 <= alpha <= 1.0):
            raise ValueError("alpha must be a finite number in [0, 1]")

        if alpha == 0.0:
            return 1.0
        if alpha == 1.0:
            return 0.0

        if beta_rounds_up:
            p = inf_mul(inf_sub(1.0, alpha), 2.0)  #
            p = inf_cast_f32(p)
        else:
            p = neg_inf_mul(neg_inf_sub(1.0, alpha), 2.0)  #
            p = neg_inf_cast_f32(p)
        p = min(p, p_max)
        if p == 0.0:
            return 0.0
```

```
30
31          if t_rounds_up:
32              y = next_up_f32(inf_cast_f32(erfc_inv(p)))  #
33              q = inf_div(mu, sqrt_2_lo)
34              t = inf_add(y, q)
35          else:
36              y = next_down_f32(neg_inf_cast_f32(erfc_inv(p)))  #
37              q = neg_inf_div(mu, sqrt_2_hi)
38              t = neg_inf_add(y, q)
39          if t > t_max:
40              return 0.0
41
42          if beta_rounds_up:
43              t = neg_inf_cast_f32(t)
44              beta_num = min(next_up_f32(inf_cast_f32(erfc(t))), 1.0)  #
45              return inf_div(beta_num, 2.0)
46          else:
47              t = inf_cast_f32(t)
48              beta_num = max(next_down_f32(neg_inf_cast_f32(erfc(t))), 0.0)  #
49              return neg_inf_div(beta_num, 2.0)
50
51      return TradeoffCurve(tradeoff)
```

Postcondition

Theorem 1.1. Given $\mu \geq 0$, the pseudocode returns a conservative implementation, with respect to intent I, of the Gaussian differential privacy tradeoff curve

$$\beta(\alpha) = \Phi\big(\Phi^{-1}(1-\alpha) - \mu\big)\,.$$

For `Reporting`, the returned $\tilde{\beta}(\alpha)$ is no larger than the exact $\beta(\alpha)$. For `Calibration`, the returned $\tilde{\beta}(\alpha)$ is no smaller than the exact $\beta(\alpha)$.

Proof. Using $\Phi(x) = \frac{1}{2}\operatorname{erfc}(-x/\sqrt{2})$, the GDP tradeoff curve can be written as

$$\beta(\alpha) = \tfrac{1}{2}\operatorname{erfc}\Big(\operatorname{erfc}^{-1}(2(1-\alpha)) + \mu/\sqrt{2}\Big)\,,$$

as defined in Theorem 2.5 of Dong, Roth, and Su 2022.

For `Reporting`, lines 25, 32, and the upward rounding of $\mu/\sqrt{2}$ make the final argument to erfc no smaller than the exact argument. Because erfc is decreasing, this can only decrease the exact erfc value. Line 48 then rounds the final numerator downward, so the returned $\tilde{\beta}(\alpha)$ is no larger than the exact GDP tradeoff value. This is conservative for reporting because smaller β is weaker.

For `Calibration`, the choices are reversed. Line 22 rounds the input to erfc^{-1} upward; since erfc^{-1} is decreasing, line 36 and the downward rounding of $\mu/\sqrt{2}$ make the final argument to erfc no larger than the exact argument. Because erfc is decreasing, this can only increase the exact erfc value. Line 44 then rounds the final numerator upward, so the returned $\tilde{\beta}(\alpha)$ is no smaller than the exact GDP tradeoff value. This is conservative for calibration because larger β is stronger.

The endpoint returns at $\alpha = 0$ and $\alpha = 1$ match the exact GDP tradeoff curve. □

References

Dong, Jinshuo, Aaron Roth, and Weijie J. Su (2022). "Gaussian Differential Privacy". In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 84.1, pp. 3–37. ISSN: 1369-7412.